

Programa de Introducción a la Programación

Docente a cargo: Sr. Santiago Daniel Montiel
Carga horaria: 6 hs semanales, 96 totales
Instituto responsable: Ciencias

Fundamentación

En su condición de primera materia de programación de la carrera, apunta al aprendizaje introductorio de programación imperativa.

Objetivo

Lograr una primera introducción a la programación imperativa y a la manipulación de tipos de datos, por medio de la construcción de programas sencillos.

Metodología

Sobre la base de cálculos y problemas sencillos se ven estructuras de control cíclicas y alternativas, variables, asignaciones, estructuras lógicas y programación estructurada. Se pretende así un primer acercamiento con acento en la práctica de la programación.

Régimen de aprobación

Para regularizar la materia se deben aprobar dos exámenes parciales o sus respectivos recuperatorios y un Trabajo Práctico Final. El primer examen parcial se toma hacia la mitad de la cursada y el segundo al final de la cursada. La materia se promociona con un promedio entre los parciales y el Trabajo Final mayor o igual a 7 y se debe rendir un examen final si el promedio obtenido es menor a 7 pero mayor o igual a 4.

Requisitos para rendir libre

Deben respetarse las correlatividades. Los alumnos deberán rendir el examen sobre la base del programa de la cátedra y entregar en la fecha de examen un Trabajo que previamente deben solicitar a los docentes.

Contenidos mínimos

Modelo de cómputo imperativo: variables, asignaciones y estructuras de control. Estructuras alternativas y estructuras cíclicas. Ciclos "for" y ciclos "while". Estado de un programa. Algoritmos, precondition y postcondición. Concepto de invariante de un ciclo. Concepto de programación estructurada. Funciones en un lenguaje imperativo. Pasaje de parámetros por valor y por referencia. Ejemplos y aplicaciones.



Lenguajes de programación imperativos. Lenguaje Python. Sintaxis del lenguaje. Tipos de datos primitivos y declaración de variables. Definición de estructuras de control y funciones. Implementación de programas en uno y varios archivos. Algoritmos sencillos para cálculos con números enteros. Sumatorias y productorias. Algoritmos de conteo. Algoritmos sobre sucesiones y cálculos sencillos. Buenas prácticas de programación. Listas y algoritmos sobre listas. Lenguaje C. Definición de arreglos en el lenguaje C. Introducción al uso de punteros y memoria dinámica.

Temario

El formato de las referencias bibliográficas indican [capítulo.sección] del libro propuesto como de lectura obligatoria.

Unidad 1: Modelo de cómputo imperativo. Breve introducción a la programación imperativa, algoritmos y lenguajes de programación [1.2,1.3,1.4]. Presentación del lenguaje Python [1.1,1.5]. Tipo de datos entero, cadena y float [2.1]. Variables, declaración de variables y asignaciones [2.2,2.3]. Expresiones y operadores aritméticos [2.4,2.5,2.6,2.7,2.9] y algunas funciones incluidas en Python.

Unidad 2: Estructuras de control alternativas. Variables booleanas, operadores relacionales y expresiones lógicas [4.1,4.2]. Operadores lógicos [4.3]. Sentencias "if", "else", "elif" [4.4,4.5,4.6] y condicionales anidados [4.7].

Unidad 3: Estructuras de control cíclicas. Ciclos "while" [6.1,6.2]. Estado de un programa y uso de variables como contadores, acumuladores y banderas [6.3,6.4]. Ejercitación con algoritmos de conteo, sucesiones, sumatorias, productorias y cálculos sencillos.

Unidad 4: Listas y cadenas. Tipo de datos compuestos [7.1,7.2,8.1,8.2,8.3]. Función Range() y ciclos "for" [7.3,8.5]. Algoritmos y operaciones sobre listas [8.6,8.7] y sobre cadenas [7.4,7.5,7.8]. Diferencias y similitudes básicas entre manejo de listas y cadenas [7.6,8.8,8.9,8.16].

Unidad 5: Concepto de programación estructurada. Definición de funciones propias en un lenguaje imperativo [3.6,3.7,3.8]. Encapsulado y variables locales [3.10,6.5,6.6,6.7,6.8]. Pasaje de parámetros por valor y por referencia [3.9,8.13]. Ejercitación implementando programas en uno o varios archivos.

Unidad 6. Se desarrolla un trabajo práctico, con entregas parciales. El objetivo del trabajo es el de construir una aplicación sencilla, incorporando de forma integrada los conocimientos vistos en la materia.

Bibliografía

Obligatoria

Python for Software Design: How to Think Like a Computer Scientist, un libro open source traducido al castellano. <http://openbookproject.net/thinkcs/python/spanish2e>

Opcional

Recomendada:

Aprenda A Programar Con Python, Zed A. Shaw, ANAYA, 2014.

Python 3, Los Fundamentos Del Lenguaje, Sébastien Chazalle, ENI, 2015.

Python Desde Cero, Ocádiz Mendoza Alfredo, EAE Editorial Academia Espanola, 2013.



Python 3 al Descubierto, Fernandez Montoro, Alfaomega, 2013.

Optativa:

Python para todos. Raúl González Duque - En español (Licencia Creative Commons).

<http://mundogeek.net/tutorial-python/>

Guía de Aprendizaje de Python. Tutorial oficial escrito por el autor de Python, Guido van Rossum, traducido por PyAr. <http://python.org.ar/pyar/Tutorial>.

Python no muerde. Roberto Alsina - En español (Licencia Creative Commons).

<http://nomuerde.netmanagers.com.ar/>

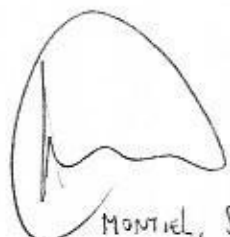
Tutor para No-Programadores. Josh Cogliati 1999-2002 - Traducción Victor M. Rosas Garcia.

http://jjc.freeshell.org/easytut/easytut_es/easytut.html

Python Instantáneo. Magnus Lie Hetland - Traducción de Marcos Sánchez Provencio.

http://users.servicios.retecal.es/tjavier/python/Pequeno_tutorial_de_Pyhton.html

Firma y aclaración del docente responsable:



MONTIEL, SANTIAGO